

This divergence similarly takes into account the distance between p and q in its derivatives through squaring. However, it is much less stable than the joint matching loss since both p and q are computed by taking the product over many small numbers. Computing the square root over this will be much less numerically stable than taking the logarithm of each individual probability and summing.

Finally, we note that we minimize the on-policy joint matching $\mathbb{E}_q[(\log p - \log q)^2]$ by taking derivatives $\mathbb{E}_q[\nabla_{\phi, \psi}(\log p - \log q)^2]$. This is technically not minimizing the joint matching, since it ignores the gradient coming from sampling from q .

F Dirichlet prior

This section describes how we fit the Dirichlet prior $p(\mathbf{P})$ used to train the inference model. During training, we keep a dataset of the last 2500 observations of $\mathbf{P} = f_{\theta}(\mathbf{x})$. We have to drop observations frequently because θ changes during training, meaning that the empirical distribution over $\mathbf{P}$ s changes as well.

We perform an MLE fit on k_W independent Dirichlet priors to get parameters α for each. The log-likelihood of the Dirichlet distribution cannot be found in closed form [39]. However, since its log-likelihood is convex, we run ADAM [25] for 50 iterations with a learning rate of 0.01 to minimize the negative log-likelihood. We refer to [39] for details on computing the log-likelihood and alternative options. Since the Dirichlet distribution accepts positive parameters, we apply the softplus function on an unconstrained parameter during training. We initialize all parameters at 0.1.

We added L2 regularization on the parameters. This is needed because at the beginning of training, all observations $\mathbf{P} = f_{\theta}(\mathbf{x})$ represent uniform beliefs over digits, which will all be nearly equal. Therefore, fitting the Dirichlet on the data will give increasingly higher parameter values, as high parameter values represent low-entropy Dirichlet distributions that produce uniform beliefs. When the Dirichlet is low-entropy, the inference models learn to ignore the input belief $\mathbf{P}$, as it never changes. The L2 regularization encourages low parameter values, which correspond to high-entropy Dirichlet distributions.

G Designing symbolic pruners

We next discuss four high-level approaches for designing the optional symbolic pruner, each with differing tradeoffs in accuracy, engineering time and efficiency:

1. **Mathematically derive efficient solvers.** For simple problems, we could mathematically derive an exact solver. One example of an efficient symbolic pruner, along with a proof for exactness, is given for Multi-digit MNISTAdd in Appendix H. This pruner is linear-time. However, for most problems we expect the pruner to be much more computationally expensive.
2. **Use SAT-solvers.** Add the sampled symbols $\mathbf{y}$ and $\mathbf{w}_{1:i}$ to a CNF-formula, and ask an SAT-solver if there is an extension $\mathbf{w}_{i+1:k_W}$ that satisfies the CNF-formula. SAT-solvers are a general approach that will work with every function c , but using them comes at a cost. The first is that we would require grounding the logical representation of the problem. Furthermore, to do SAT-solving, we have to solve a linear amount of NP-hard problems. However, competitive SAT solvers can deal with substantial problems due to years of advances in their design [7], and a linear amount of NP-hard calls is a lower complexity class than #P hard. Using SAT-solvers will be particularly attractive in problem settings where safety and verifiability are critical.
3. **Prune with local constraints.** In many structured prediction tasks, we can use local constraints of the symbolic problem to prune paths that are guaranteed to lead to branches that can never create possible worlds. However, local constraints do not guarantee that each non-pruned path contains a possible world, but this does not bias the inference model, as the neural network will (eventually) learn when an expansion would lead to an unsatisfiable state. One example is the shortest path problem, where we filter out directions that would lead outside the $N \times N$ grid, or that would create cycles (See Appendix I.3). However, this only ensures we find *a* path, but does not ensure it is the shortest one.

4. **Learn the pruner.** Finally, we can learn the pruner, that is, we can train a neural network to learn satisfiability checking. One possible approach is to reuse the inference model trained on the belief $\mathbf{P}$ that uniformly distributes mass over all worlds. Learned pruners will be as quick as regular inference models, but are less accurate than symbolic pruners and will not guarantee that constraints are always satisfied during test-time. We leave experimenting with learning the pruner for future work.

H MNISTAdd Symbolic Pruner

In this section, we describe a symbolic pruner for the Multi-digit MNISTAdd problem, which we compute in time linear to N . Note that $\mathbf{w}_{1:N}$ represents the first number and $\mathbf{w}_{N+1:2N}$ the second. We define $n_1 = \sum_{i=1}^N w_i \cdot 10^{N-i-1}$ and $n_2 = \sum_{i=1}^N w_{N+i} \cdot 10^{N-i-1}$ for the integer representations of these numbers, and $y = \sum_{i=1}^{N+1} y_i \cdot 10^{N-i}$ for the sum label encoded by $\mathbf{y}$. We say that partial generation $\mathbf{w}_{1:k}$ has a *completion* if there is a $\mathbf{w}_{k+1:2N} \in \{0, \dots, 9\}^{2N-k}$ such that $n_1 + n_2 = y$.

Proposition H.1. *For all $N \in \mathbb{N}$, $\mathbf{y} \in \{0, 1\} \times \{0, \dots, 9\}^N$ and partial generation $\mathbf{w}_{1:k-1} \in \{0, \dots, 9\}^k$ with $k \in \{1, \dots, 2N\}$, the following algorithm rejects all w_k for which $\mathbf{w}_{1:k}$ has no completions, and accepts all w_k for which there are:*

- $k \leq N$: Let $l_k = \sum_{i=1}^{k+1} y_i \cdot 10^{k+1-i}$ and $p_k = \sum_{i=1}^k w_k \cdot 10^{k-i}$. Let $S = 1$ if $k = N$ or if the $(k+1)$ th to $(N+1)$ th digit of y are all 9, and $S = 0$ otherwise. We compute two boolean conditions for all $w_k \in \{0, \dots, 9\}$:

$$0 \leq l_k - p_k \leq 10^k - S \quad (34)$$

We reject all w_k for which either condition does not hold.

- $k > N$: Let $n_2 = y - n_1$. We reject all $w_k \in \{0, \dots, 9\}$ different from $w_k = \lfloor \frac{n_2}{10^{N-k-1}} \rfloor \bmod 10$, and reject all w_k if $n_2 < 0$ or $n_2 \geq 10^N$.

Proof. For $k > N$, we note that n_2 is fixed given y and n_1 through linearity of summation, and we only consider $k \leq N$. We define $a_k = \sum_{i=k+2}^{N+1} y_i \cdot 10^{N+1-i}$ as the sum of the remaining digits of y . We note that $y = l_k \cdot 10^{N-k} + a_k$.

Algorithm rejects w_k without completions We first show that our algorithm only rejects w_k for which no completion exists. We start with the constraint $0 \leq l_k - p_k$, and show that whenever this constraint is violated (i.e., $p_k > l_k$), $\mathbf{w}_{1:k}$ has no completion. Consider the smallest possible completion of $\mathbf{w}_{k+1:N}$: setting each to 0. Then $n_1 = p_k \cdot 10^{N-k}$. First, note that

$$10^{N-k} > 10^{N-k} - 1 \geq a_k$$

Next, add $l_k \cdot 10^{N-k}$ to both sides

$$(l_k + 1) \cdot 10^{N-k} > l_k \cdot 10^{N-k} + a_k = y$$

By assumption, p_k is an integer upper bound of l_k and so $p_k \geq l_k + 1$. Therefore,

$$n_1 = p_k \cdot 10^{N-k} > y$$

Since n_1 is to be larger than y , n_2 has to be negative, which is impossible.

Next, we show the necessity of the second constraint. Assume the constraint is unnecessary, that is, $l_k > p_k + 10^k - S$. Consider the largest possible completion $\mathbf{w}_{k+1:N}$ by setting each to 9. Then

$$\begin{aligned} n_1 &= p_k \cdot 10^{N-k} + 10^{N-k} - 1 \\ &= (p_k + 1) \cdot 10^{N-k} - 1 \end{aligned}$$

We take n_2 to be the maximum value, that is, $n_2 = 10^N - 1$. Therefore,

$$n_1 + n_2 = 10^N - (p_k + 1) \cdot 10^{N-k} - 2$$

We show that $n_1 + n_2 < y$. Since we again have an integer upper bound, we know $l_k \geq p_k + 10^k - S + 1$. Therefore,

$$\begin{aligned} y &\geq (p_k + 1 + 10^k - S)10^{N-k} + a_k \\ &\geq n_1 + n_2 + 2 - S \cdot 10^{N-k} + a_k \end{aligned}$$

There are two cases.

- $S = 0$. Then $a_k < 10^{N-k} - 1$, and so

$$y \geq n_1 + n_2 + 2 + a_k > n_1 + n_2.$$

- $S = 1$. Then $a_k = 10^{N-k} - 1$, and so

$$y \geq n_1 + n_2 + 1 > n_1 + n_2.$$

Algorithm accepts w_k with completions Next, we show that our algorithm only accepts w_k with completions. Assume Equation 34 holds, that is, $0 \leq l_k - p_k \leq 10^k - S$. We first consider all possible completions of $w_{1:k}$. Note that $p_k \cdot 10^{N-k} \leq n_1 \leq p_k \cdot 10^{N-k} + 10^{N-k} - 1$ and $0 \leq n_2 \leq 10^N - 1$, and so

$$p_k \cdot 10^{N-k} \leq n_1 + n_2 \leq (p_k + 1) \cdot 10^{N-k} + 10^N - 2.$$

Similarly,

$$l_k \cdot 10^{N-k} \leq y \leq (l_k + 1) \cdot 10^{N-k} - 1.$$

By assumption, $p_k \leq l_k$, so $p_k \cdot 10^{N-k} \leq l_k \cdot 10^{N-k}$. For the upper bound, we again consider two cases. We use the second condition $l_k \leq 10^k + p_k - S$:

- $S = 0$. Then (since there are no trailing 9s),

$$\begin{aligned} y &\leq (l_k + 1) \cdot 10^{N-k} - 2 \\ &\leq (10^k + p_k + 1) \cdot 10^{N-k} - 1 \\ &= (p_k + 1) \cdot 10^{N-k} + 10^N - 2. \end{aligned}$$

- $S = 1$. Then with trailing 9s,

$$\begin{aligned} y &= (l_k + 1) \cdot 10^{N-k} - 1 \\ &\leq (10^k + p_k) \cdot 10^{N-k} - 1 \\ &= p_k \cdot 10^{N-k} + 10^N - 1 \\ &\leq (p_k + 1) \cdot 10^{N-k} + 10^N - 2, \end{aligned}$$

since $10^{N-k} \geq 1$.

Therefore,

$$p_k \cdot 10^{N-k} \leq y \leq (p_k + 1) \cdot 10^{N-k} + 10^N - 2$$

and so there is a valid completion. □

I Details of the experiments

I.1 Multi-digit MNISTAdd

Like [36, 37], we take the MNIST [30] dataset and use each digit exactly once to create data. We follow [36] and require more unique digits for increasing N . Therefore, the training dataset will be of size $60000/2N$ and the test dataset of size $10000/2N$.

I.1.1 Hyperparameters

We performed hyperparameter tuning on a held-out validation set by splitting the training data into 50,000 and 10,000 digits, and forming the training and validation sets from these digits. We progressively increased N from $N = 1$, $N = 3$, $N = 4$ to $N = 8$ during tuning to get improved insights into what hyperparameters are important. The most important parameter, next to learning rate, is the weight of the L2 regularization on the Dirichlet prior’s parameters which should be very high. We used ADAM [25] throughout. We ran each experiment 10 times to estimate average accuracy, where each run computes 100 epochs over the training dataset. We used Nvidia RTX A4000s GPUs and 24-core AMD EPYC-2 (Rome) 7402P CPUs.